- Semáforo

- Mutex

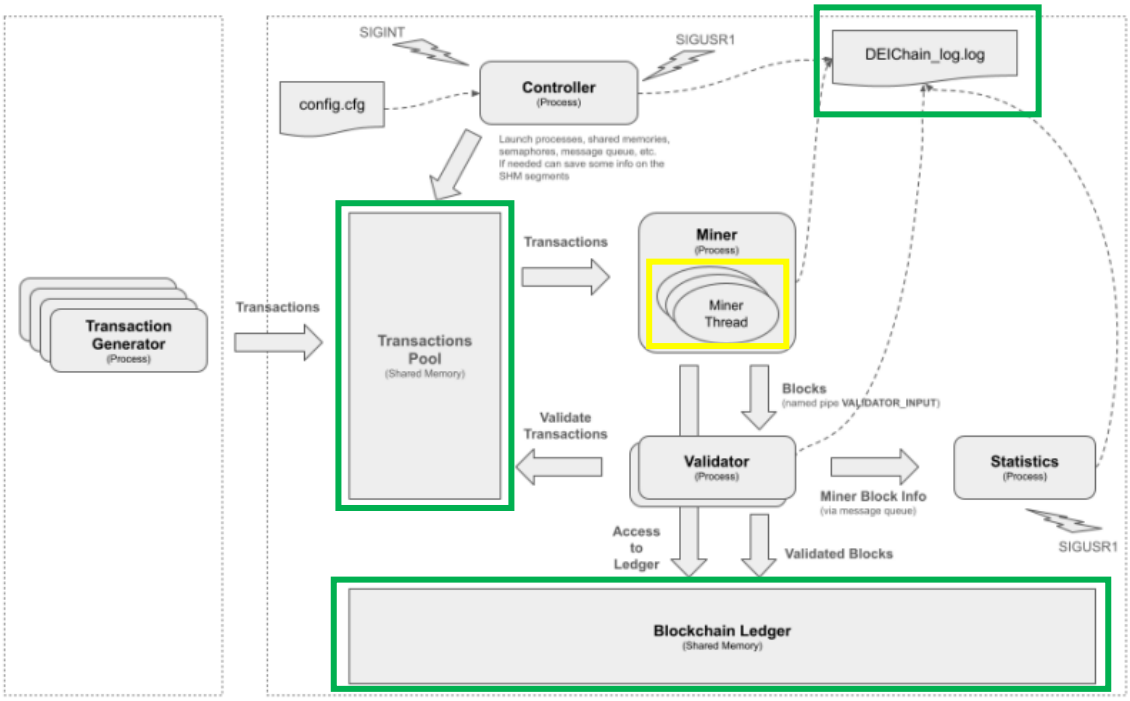



*Figure 1. DEIChain modules description and interaction*

Shared memory(Transactions Pool) - Para que a informação a ser alterada pelo Transaction Generator e pelo Validator não entre em conflito quando estes estiverem a aceder à Shared Memory estamos a usar Semáforos para que não ocorram esses conflitos.

Shared memory(Blockchain Ledger) - Para que a informação a ser alterada pelo Validator não entre em conflito quando este estiver a aceder à Shared Memory estamos a usar Semáforos para que não ocorram esses conflitos.

Log file - Para que este ficheiro não seja acedido por mais do que um processo ao mesmo tempo foi criado um semáforo para que as linhas escritas no ficheiro não sejam corrompidas.

Miner – Para que as diferentes threads do miner não estejam a ler da shared memory ao mesmo tempo usámos um mutex para evitar eventuais erros que possam acontecer

Além dos mecanismos de sincronização mostrados na figura acima, optámos também  por usar um semáforo para sincronizar os prints, de modo a não haver sobreposição entres os vários processos.

De forma a cumprir os objetivos pretendidos neste projeto tomamos algumas decisões não inclusas no enunciado do mesmo:

- Criação de uma thread auxiliar tanto no controler.c como no statistics.c de forma a poder detetar o sinal SIGUSR1, no caso deste projeto este sinal é o Control+L;
- Escolha random de transações para a criação de blocos, permitindo haver um equilibrio entre as rewards totais de cada bloco;
- Passagem por parametro do tamanho de cada bloco do controler para o validator de modo a que o segundo saiba o tamanho necessário que terá que ler do pipe;
- Criação de uma função responsável por ordenar as estatísticas pelo id do miner no statistics.c
- Verificação da hash, aceitando apenas blocos em que a hash comece exatamente pelo mesmo número de zeros que a maior reward presente no bloco;
- Utilização de um ciclo while para garatir que não haja leituras parciais da informação contida no pipe.

Para o melhor funcionamento do programa decidimos criar alguma estruturas de modo a armazenar os dados mais importantes de certos objetos:

- transaction- estrutura responsável por guardar os dados de cada transação. Estes dados são: id, reward, valor, tempo de criação, idade , empty, sendo a ultima resposável por nos dizer se dado espaço na shared meory está ocupado por uma transação ou não;
- MinerStats- estrutura resposável por guardar os dados das estatísticas que são referentes a cada miner. Estes dados são: id do miner, número de blocos válidos, numero de inválidos e a recompensa total;
- msg- - estrutura resposável por facilitar a escrita nas mensagens na message Queue
- transactions_Pool- estrutura usada na shared memory que guarda as transações;
- block- estrutura resposável por guardar os dados de cada bloco
- blockchain_Ledger- estrutura usada na shared memory que guarda os blocos válidos.

Tempo dispendido para realização deste projeto:
Cláudio Catarino – 65h+

Miguel Calado – 65h+